

Neural Networks and Deep Learning

Dr.Varodom Toochinda
EMME, Kasetsart University

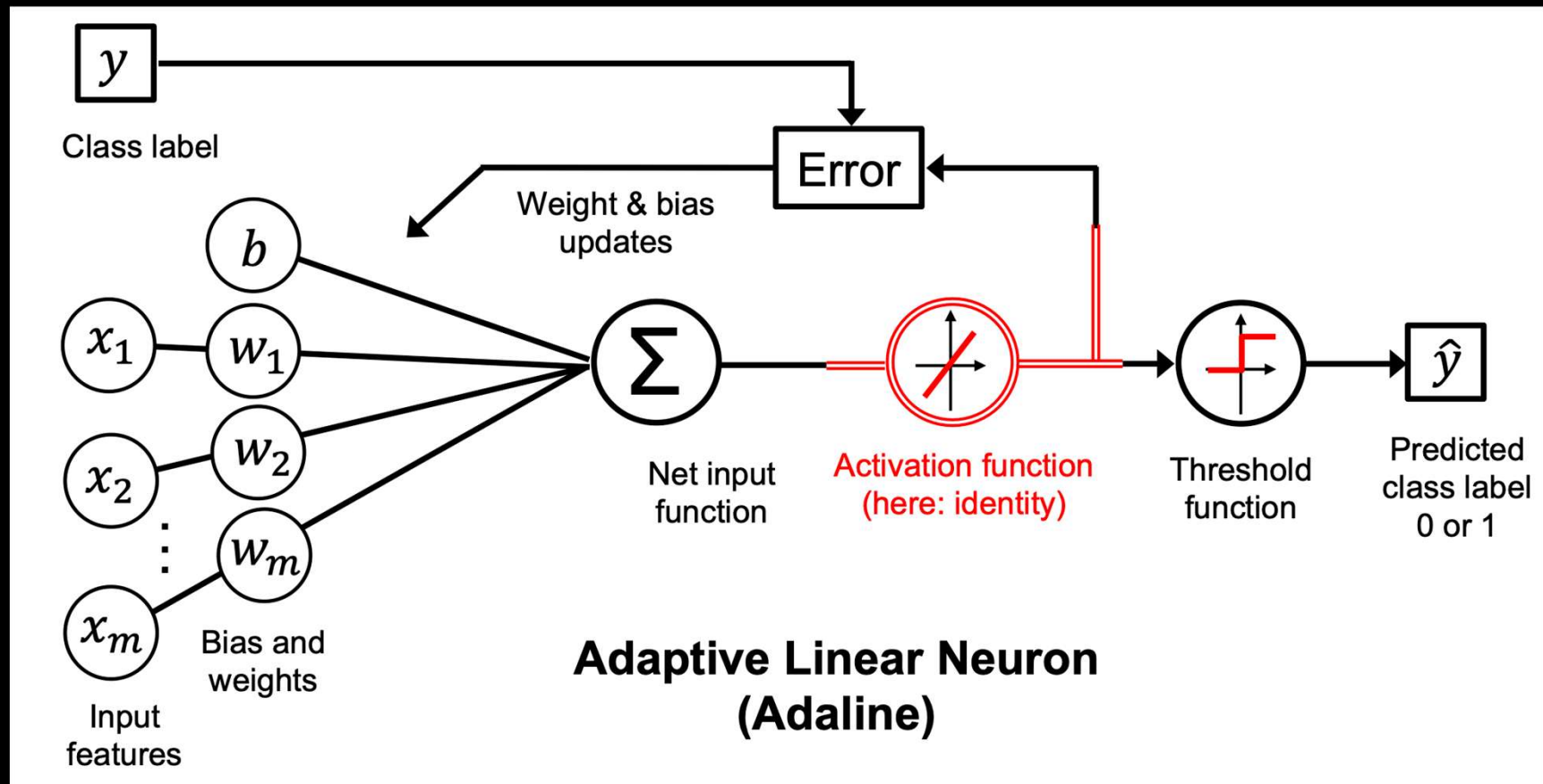
Topics

Implementing an NN
model from scratch

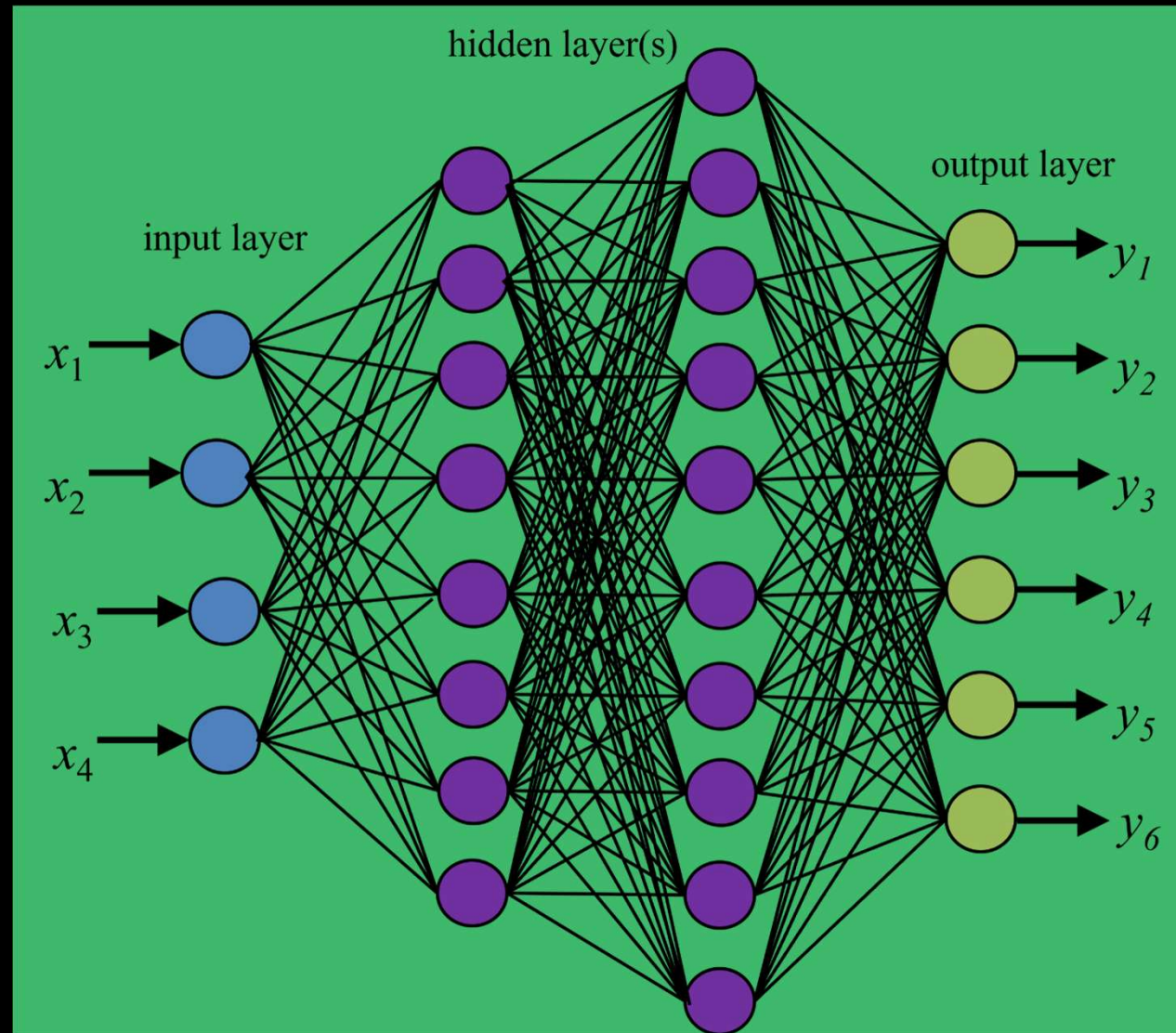
Building an NN
model in PyTorch

The mechanics of
PyTorch (optional)

Recall the Adaline



Multi-layer NN (MLP or DNN)

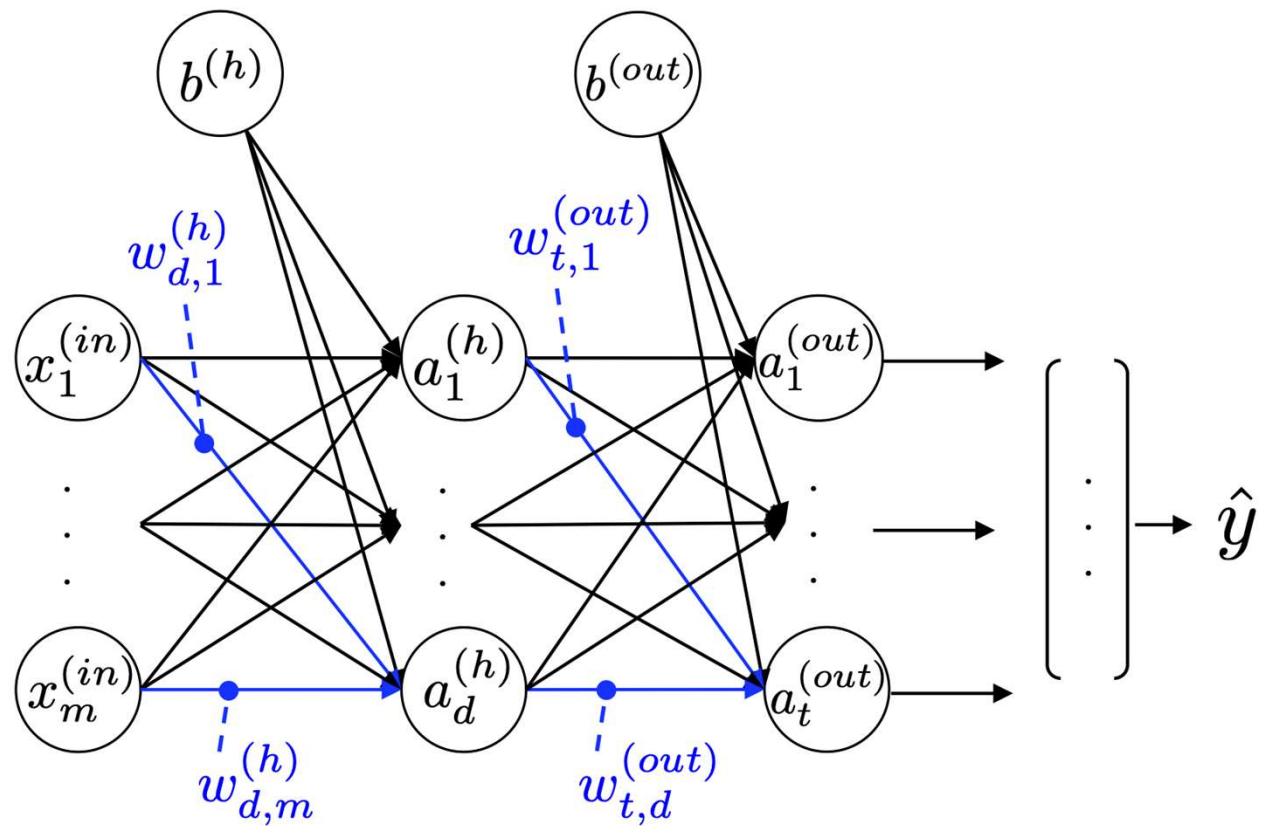


Universal Approximation Theorem

In the field of machine learning, the universal approximation theorems state that neural networks with a certain structure can, **in principle**, approximate any continuous function to any desired degree of accuracy. These theorems provide a mathematical justification for using neural networks, assuring researchers that a sufficiently large or deep network can model the complex, non-linear relationships often found in real-world data.

https://en.wikipedia.org/wiki/Universal_approximation_theorem

A two-layer MLP



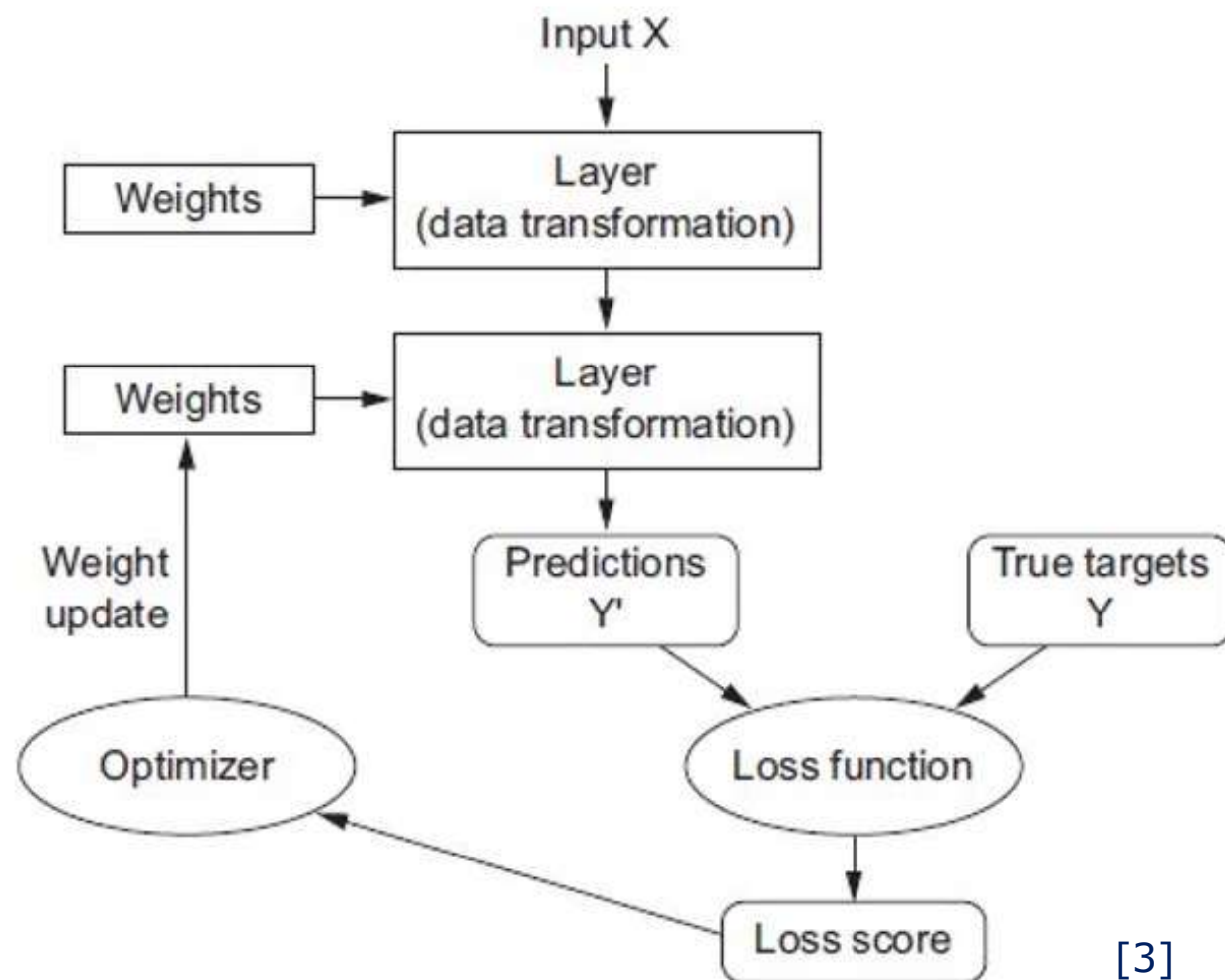
From
Figure 11.2 of [1]

Data input
("input layer" in)

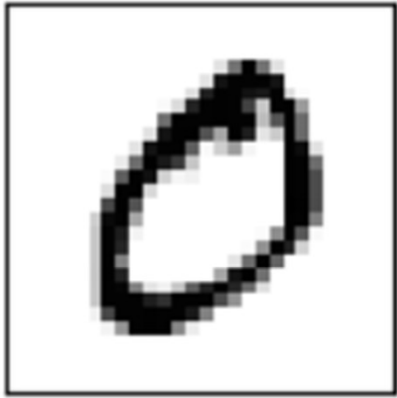
1st layer
(hidden layer h)

2nd layer
(output layer out)

MLP learning structure



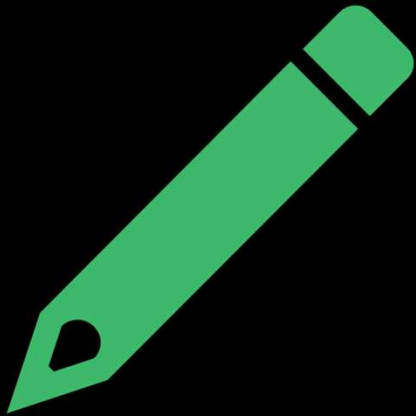
[3]



Run code in [mlpi25_lect09.ipynb](#)

Classifying MNIST hand- written digits

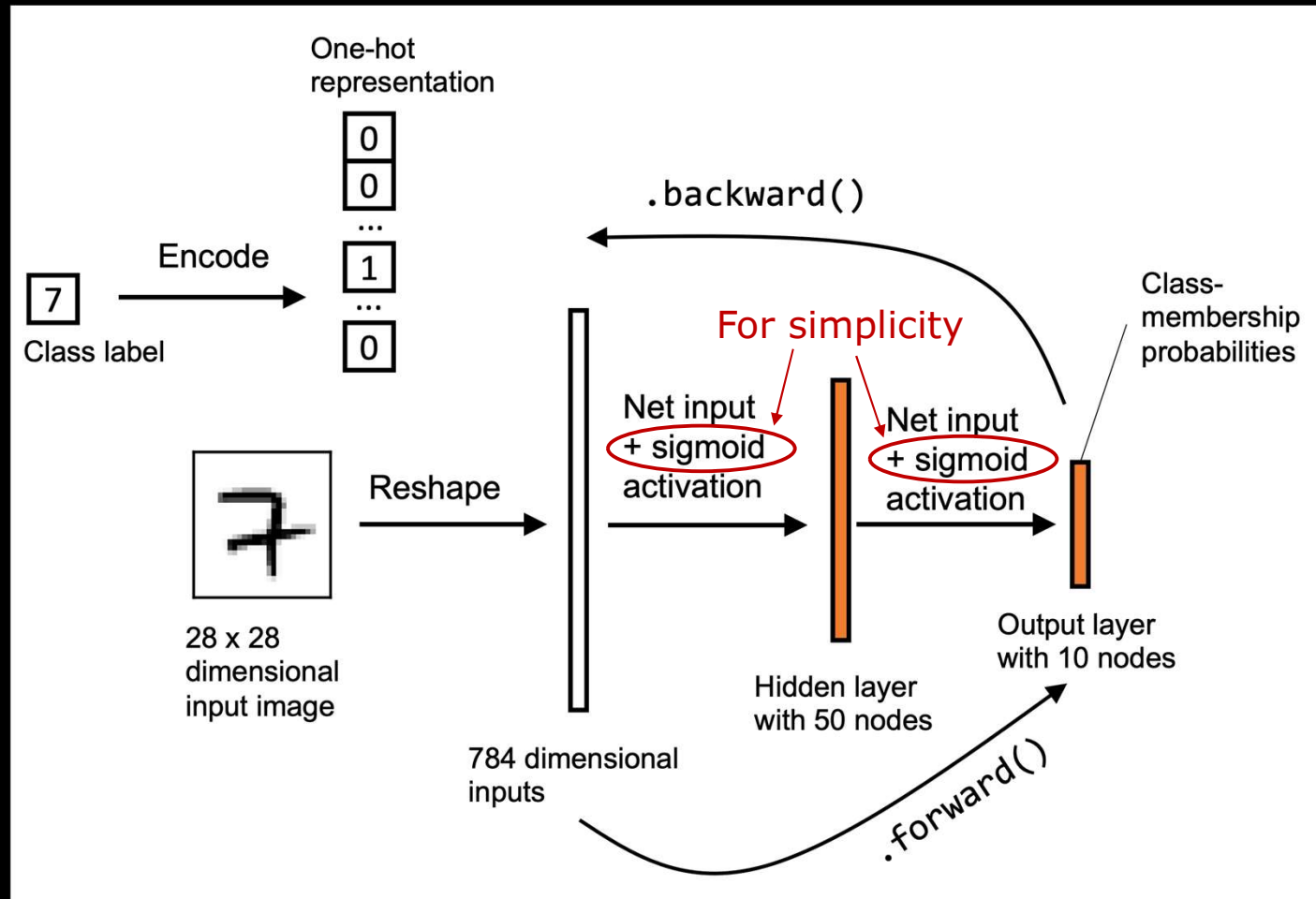




Implementing an MLP

- See NeuralNetMLP class in [mlpi25_lect09.ipynb](#)
- 3 class methods:
 - `__init__()` # constructor
 - `forward()` # prediction
 - `backward()` # compute gradients
- For simplicity, MSE loss is used in `backward()`
- Detailed explanation in [1]

NN architecture for labeling hand-written digits



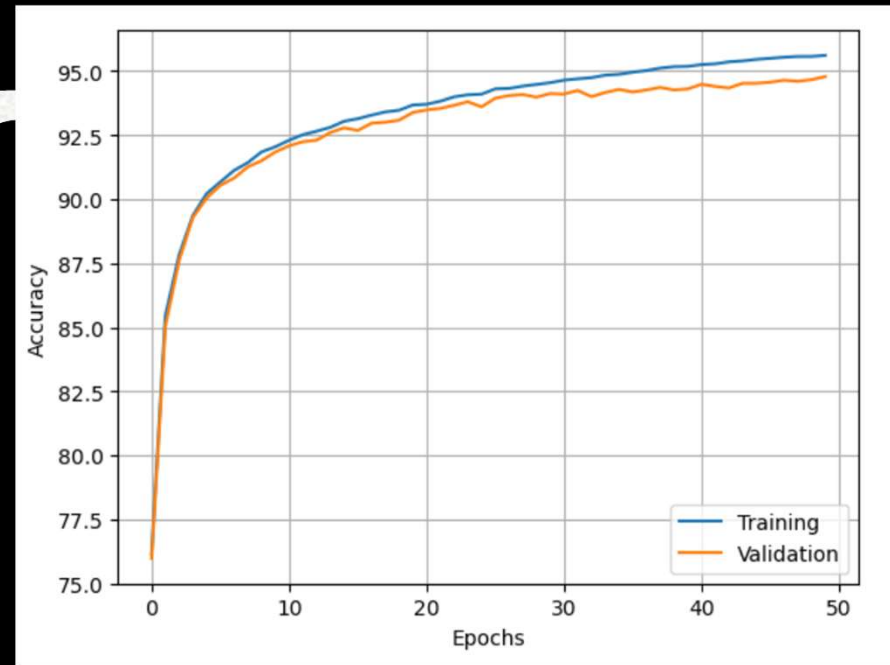
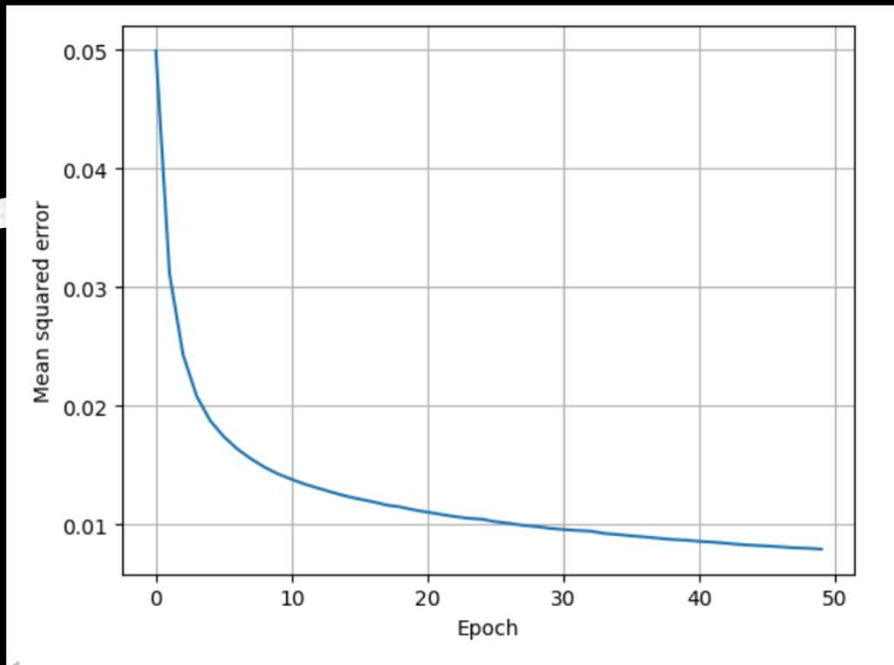
From
Figure 11.6 of [1]

```
epoch_loss, epoch_train_acc, epoch_valid_acc = train(  
    model, X_train, y_train, X_valid, y_valid,  
    num_epochs=50, learning_rate=0.1)
```

[16] 2m 10.2s

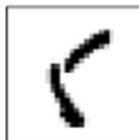
... Epoch: 001/050 | Train MSE: 0.05 | Valid MSE: 0.05 | Train Acc: 76.15% | Valid Acc: 75.98%
Epoch: 002/050 | Train MSE: 0.03 | Valid MSE: 0.03 | Train Acc: 85.45% | Valid Acc: 85.04%
Epoch: 003/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 87.82% | Valid Acc: 87.60%
Epoch: 004/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 89.36% | Valid Acc: 89.28%
Epoch: 005/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 90.21% | Valid Acc: 90.04%
Epoch: 006/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 90.67% | Valid Acc: 90.54%
Epoch: 007/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 91.12% | Valid Acc: 90.82%
Epoch: 008/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 91.43% | Valid Acc: 91.26%
Epoch: 009/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 91.84% | Valid Acc: 91.57%
Epoch: 010/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 92.04% | Valid Acc: 91.78%
Epoch: 011/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 92.30% | Valid Acc: 91.99%
Epoch: 012/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 92.51% | Valid Acc: 92.19%
Epoch: 013/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 92.67% | Valid Acc: 92.39%
Epoch: 014/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 92.80% | Valid Acc: 92.59%

Evaluating NN performance

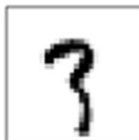


Misclassification images

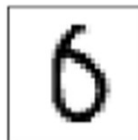
1) True: 5
Predicted: 0



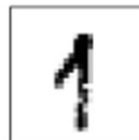
2) True: 3
Predicted: 7



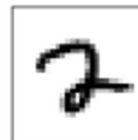
3) True: 0
Predicted: 5



4) True: 1
Predicted: 9



5) True: 2
Predicted: 7



6) True: 4
Predicted: 8



7) True: 5
Predicted: 9



8) True: 5
Predicted: 3



9) True: 8
Predicted: 5



10) True: 5
Predicted: 4



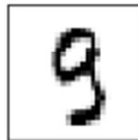
11) True: 5
Predicted: 7



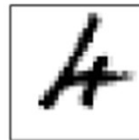
12) True: 5
Predicted: 0



13) True: 9
Predicted: 3



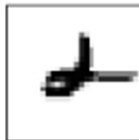
14) True: 4
Predicted: 6



15) True: 9
Predicted: 0



16) True: 2
Predicted: 4



17) True: 9
Predicted: 5



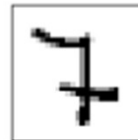
18) True: 0
Predicted: 6



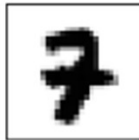
19) True: 8
Predicted: 5



20) True: 7
Predicted: 2



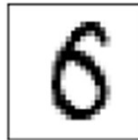
21) True: 7
Predicted: 2



22) True: 8
Predicted: 1



23) True: 6
Predicted: 5



24) True: 9
Predicted: 4



25) True: 2
Predicted: 8





Optional reading for Chapter 11 of [1]

- Training an artificial neural network (p.360)
 - Computing the loss function
 - Developing your understanding of backpropagation
 - Training neural networks via backpropagation
 - About convergence in neural networks

Building NN model in PyTorch

<https://docs.pytorch.org/tutorials/beginner/basics/intro.html>

 Run in Google Colab

 PyTorch

Learn the Basics || [Quickstart](#) || [Tensors](#) || [Datasets & Loaders](#) || [Save & Load Model](#)

Learn the Basics

Created On: Feb 09, 2021 | Last Updated: Jul 07, 2025 | Last Verified: Jul 07, 2025

Authors: [Suraj Subramanian](#), [Seth Juarez](#), [Cassie Breivik](#)

Most machine learning workflows involve working with pre-trained models. This tutorial introduces you to a collection of PyTorch models that can be used to build your own machine learning applications.

Topics

Tensors in PyTorch

Building input pipelines

Building an NN model

Choosing activation functions in MLP

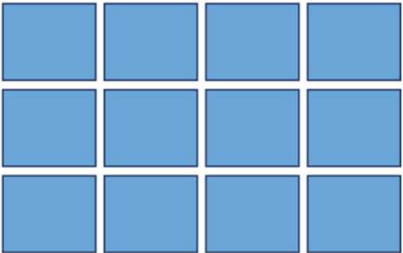
The Mechanics of PyTorch (optional) see [mlpi25_lect09A.ipynb](#)

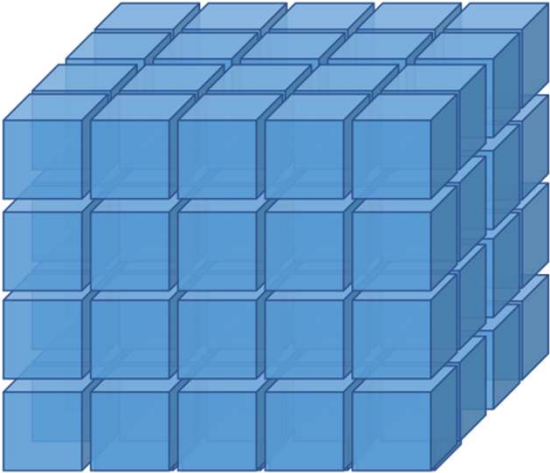
Ref: chapter 12 – 13 of [1]

Different types of tensor in PyTorch

Rank 0: 
(scalar)

Rank 1: 
(vector)

Rank 2: (matrix)


Rank 3: 

Tensor creation/operations in PyTorch

- See [mlpi25_lect09.ipynb](#)

Building input pipelines in PyTorch

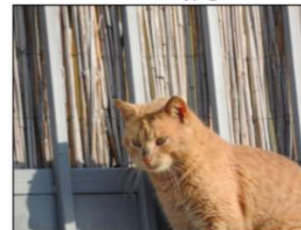
- Creating a PyTorch DataLoader
 - `torch.utils.data.DataLoader()`
- Combining two tensors into a joint dataset
 - `torch.utils.data.TensorDataset()`
- Shuffle, batch, and repeat
- Creating a dataset from files

Example images

cat-01.jpg



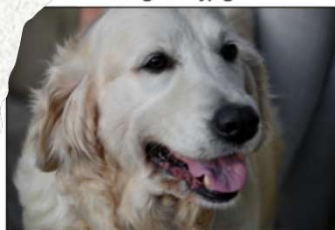
cat-02.jpg



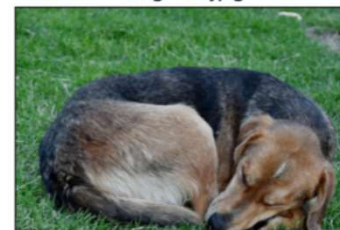
cat-03.jpg



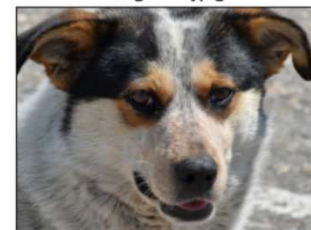
dog-01.jpg



dog-02.jpg



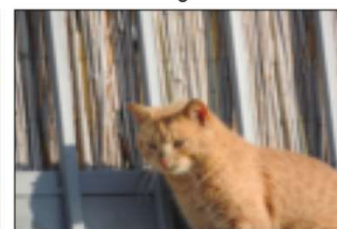
dog-03.jpg



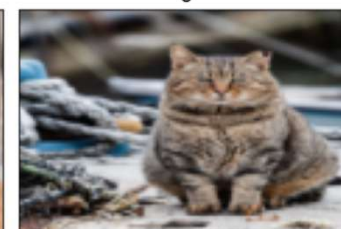
0



0



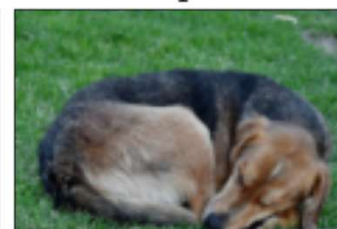
0



1



1



1



Fetching from torchvision.datasets

- CelebA

- <http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html>

- torchvision.datasets.CelebA

- <https://pytorch.org/vision/stable/datasets.html#celeba>

Note : this dataset is large (1.44 G) and may cause technical difficulty downloading.
See [1] for alternative way to get the files.

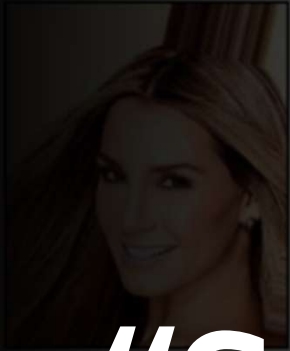
- MNIST

- torchvision.datasets.MNIST

- <https://pytorch.org/vision/stable/datasets.html#mnist>

“Smiling” celeb

1



1



0



0



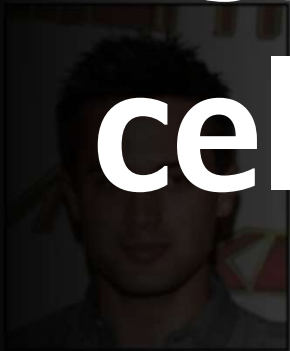
0



0



0



1



0



1



1



1



1



0



1



1



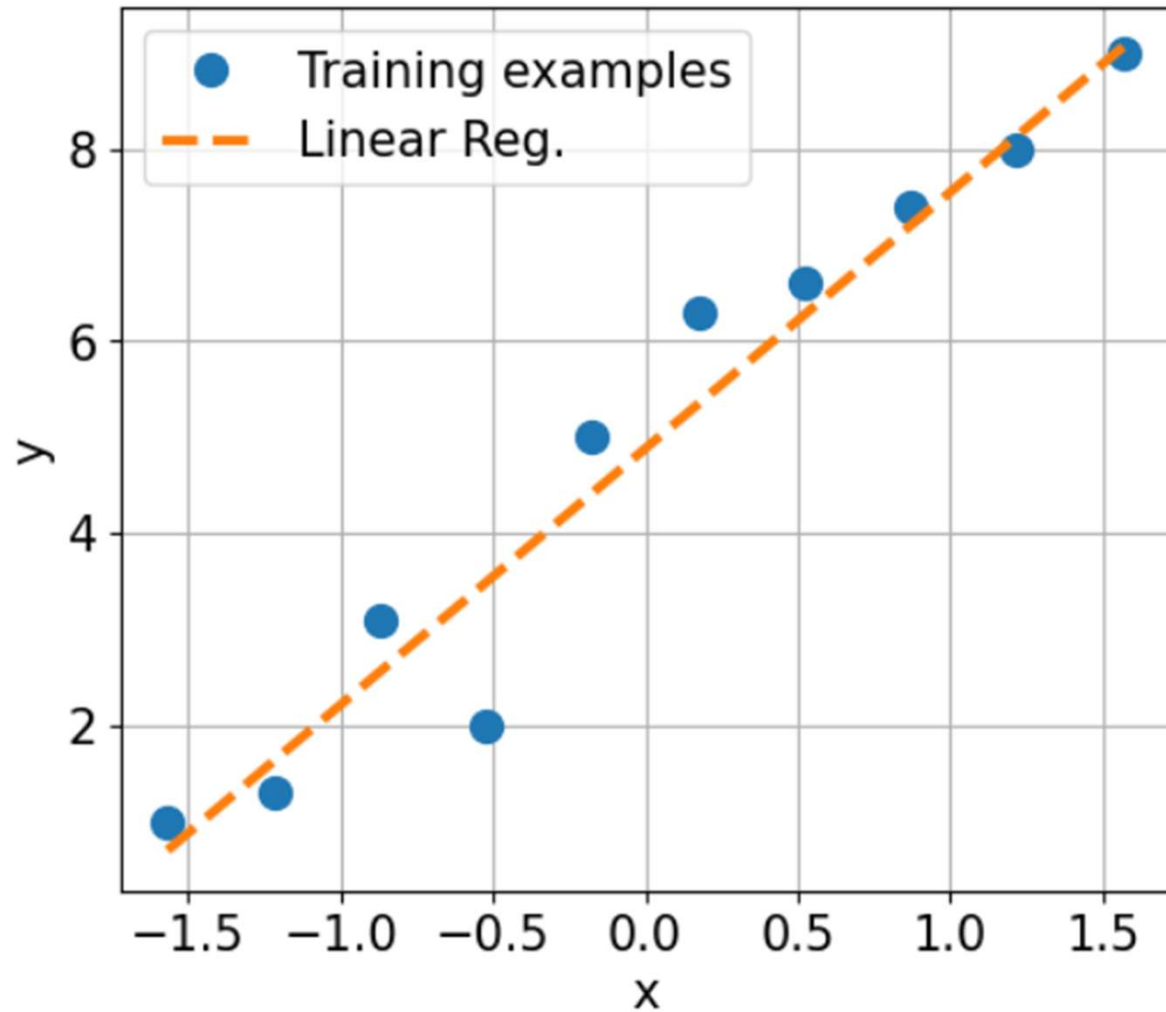
1



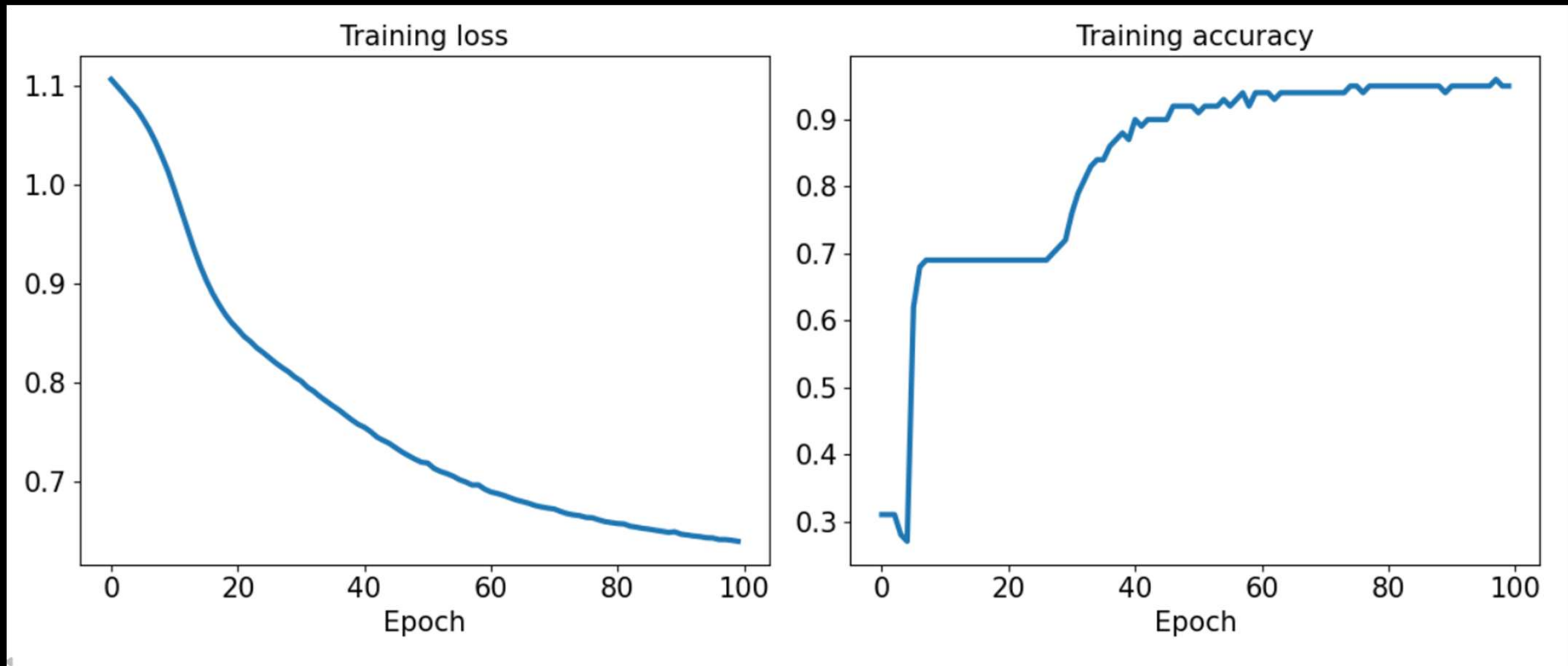
Building NN model

- linear regression
- MLP

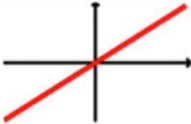
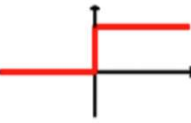
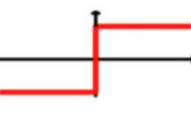
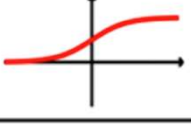
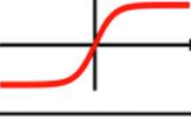
Linear regression model



MLP for iris dataset



Activation functions

Activation function	Equation	Example	1D graph
Linear	$\sigma(z) = z$	Adaline, linear regression	
Unit step (Heaviside function)	$\sigma(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\sigma(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise linear	$\sigma(z) = \begin{cases} 0 & z \leq -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \leq z \leq \frac{1}{2} \\ 1 & z \geq \frac{1}{2} \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\sigma(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, multilayer NN	
Hyperbolic tangent (tanh)	$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\sigma(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

The mechanics of PyTorch (optional reading)

See chapter 13 of [1] and [mlpi25_lect9A.ipynb](#)

References



1. S. Raschka, Y. Liu and V. Mirjalili. Machine Learning with PyTorch and Scikit-Learn.
<https://sebastianraschka.com/blog/2022/ml-pytorch-book.html> Packt Publishing. 2022.



2. A. Ng and T. Ma. CS229 Lecture Notes.
https://cs229.stanford.edu/main_notes.pdf Stanford University. 2023.



3. F. Chollet. Deep Learning with Python 2ed. Manning Publications Co. 2021.



4. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE, 86(11): 2278-2324, 1998.



5. C.M. Bishop. Neural Networks for Pattern Recognition, Oxford University Press, 1995.